Experiment 7: House Price Prediction using Linear Regression

Aim

To build a predictive model using Linear Regression to estimate house prices based on given features, and evaluate the model performance using metrics like Mean Squared Error (MSE) and R-squared.

Code

```
# EX 7: HOUSE PRICE PREDICTION USING LINEAR REGRESSION

# 1. Importing Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 2. Loading the Data
df = pd.read_csv("Housing.csv")

# 3. Exploratory Data Analysis (EDA)
print(df.head())
print(df.info())
print(df.describe())

# 4. Checking for Missing Values
print(df.isnull().sum())

# 5. Outlier Analysis
fig, axs = plt.subplots(2, 3, figsize=(10, 5))
sns.boxplot(y=df['price'], ax=axs[0, 0])
sns.boxplot(y=df['area'], ax=axs[0, 1])
sns.boxplot(y=df['bedrooms'], ax=axs[0, 2])
sns.boxplot(y=df['bathrooms'], ax=axs[1, 0])
sns.boxplot(y=df['stories'], ax=axs[1, 1])
sns.boxplot(y=df['parking'], ax=axs[1, 2])
plt.tight_layout()
plt.show()

# 6. Pair Plot
sns.pairplot(df)
plt.show()

# 7. Data Preprocessing
```

```
features = ['mainroad', 'guestroom', 'basement', 'hotwaterheating', 'airconditioning', 'prefarea']
df_encoded = pd.get_dummies(df[features], drop_first=True)

# Adding Target Variable
df_encoded['price'] = df['price']

# 8. Splitting the Data
X = df_encoded.drop('price', axis=1)
y = df_encoded['price']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 9. Building the Regression Model
model = LinearRegression()
model.fit(X_train, y_train)

# 10. Making Predictions
y_pred = model.predict(X_test)

# 11. Evaluating the Model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse}")
print(f"R-squared: {r2}")

# 12. Visualization of Actual vs Predicted Prices
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2)
plt.title('Actual vs Predicted Prices')
plt.xlabel('Actual Prices')
plt.ylabel('Predicted Prices')
plt.show()

# 13. Residual Plot
plt.figure(figsize=(10, 6))
sns.residplot(x=y_pred, y=y_test - y_pred, lowess=True)
plt.title('Residuals vs Fitted')
plt.xlabel('Fitted values')
plt.ylabel('Residuals')
plt.axhline(0, linestyle='--')
plt.show()
```

Output

```
    price  area  bedrooms  bathrooms  stories mainroad guestroom basement  \
```

```
0  13300000  7420       4        2       3     yes       no       no
1  12250000  8960       4        4       4     yes       no       no
2  12250000  9960       3        2       2     yes       no      yes
3  12215000  7500       4        2       2     yes       no      yes
4  11410000  7420       4        1       2     yes      yes      yes


  hotwaterheating airconditioning  parking prefarea furnishingstatus
0              no              yes        2      yes        furnished
1              no              yes        3       no        furnished
2              no               no        2      yes   semi-furnished
3              no              yes        3      yes        furnished
4              no              yes        2       no        furnished
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   price             545 non-null    int64
 1   area              545 non-null    int64
 2   bedrooms          545 non-null    int64
 3   bathrooms         545 non-null    int64
 4   stories           545 non-null    int64
 5   mainroad          545 non-null    object
 6   guestroom         545 non-null    object
 7   basement          545 non-null    object
 8   hotwaterheating   545 non-null    object
 9   airconditioning   545 non-null    object
 10  parking           545 non-null    int64
 11  prefarea          545 non-null    object
 12  furnishingstatus  545 non-null    object
dtypes: int64(6), object(7)
memory usage: 55.5+ KB
```

None

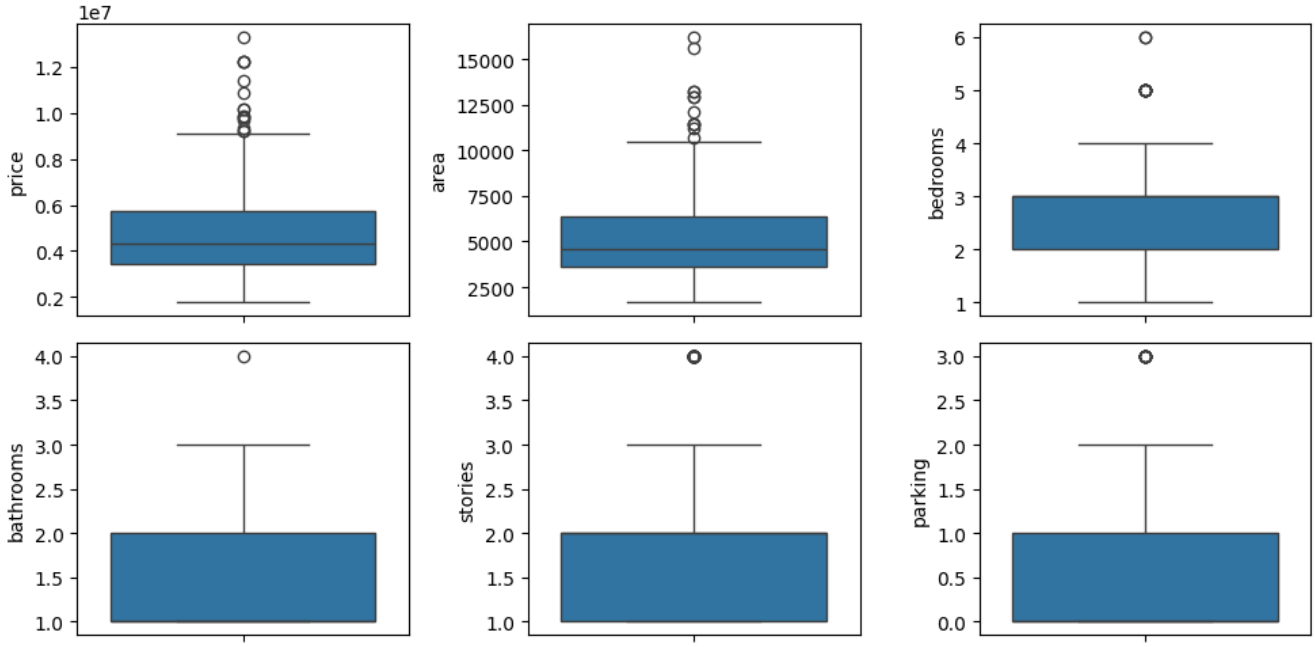
	price	area	bedrooms	bathrooms	stories	\
count	5.450000e+02	545.000000	545.000000	545.000000	545.000000	
mean	4.766729e+06	5150.541284	2.965138	1.286239	1.805505	
std	1.870440e+06	2170.141023	0.738064	0.502470	0.867492	
min	1.750000e+06	1650.000000	1.000000	1.000000	1.000000	
25%	3.430000e+06	3600.000000	2.000000	1.000000	1.000000	
50%	4.340000e+06	4600.000000	3.000000	1.000000	2.000000	
75%	5.740000e+06	6360.000000	3.000000	2.000000	2.000000	
max	1.330000e+07	16200.000000	6.000000	4.000000	4.000000	

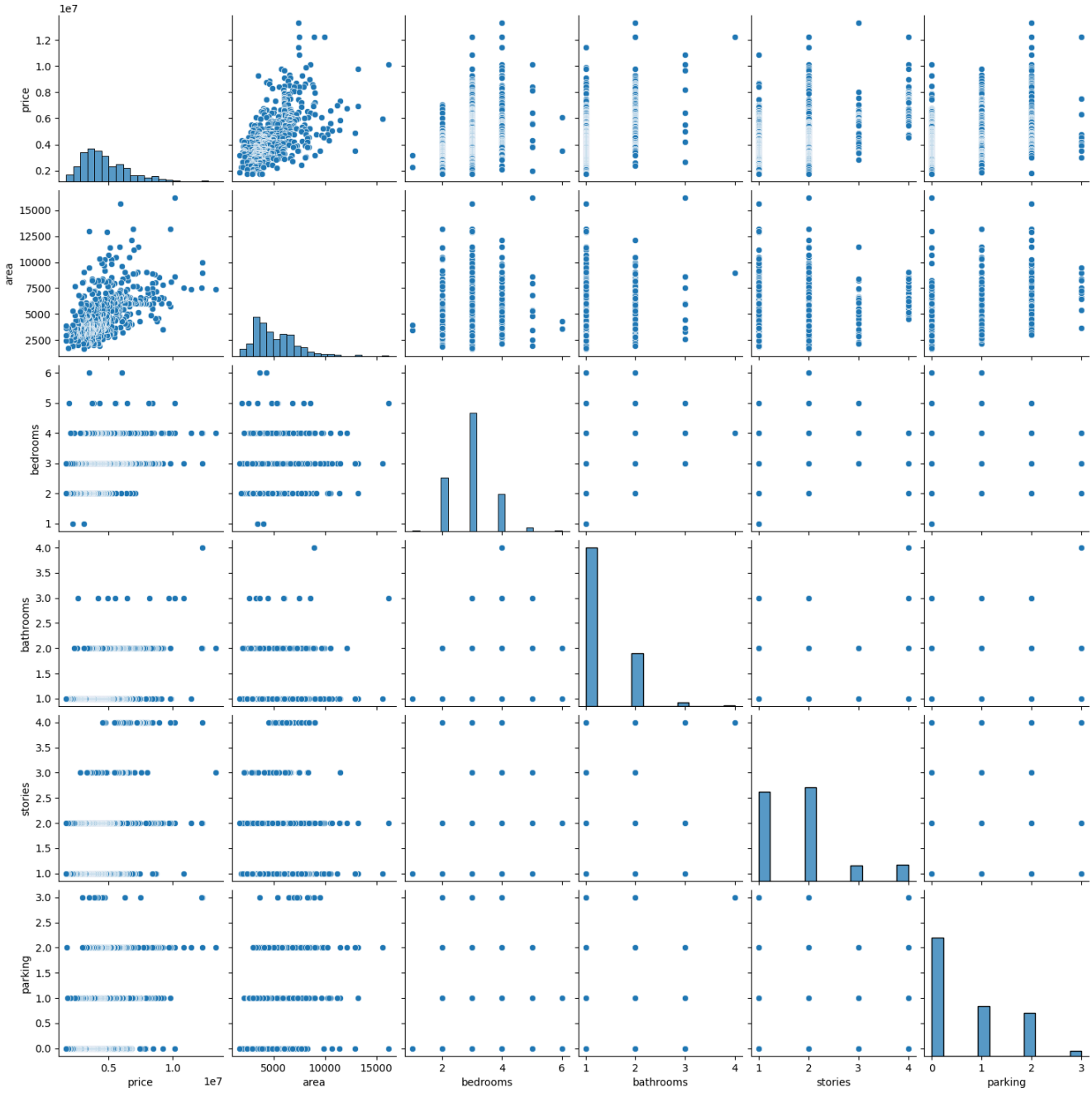
	parking
count	545.000000
mean	0.693578
std	0.861586
min	0.000000
25%	0.000000
50%	0.000000
75%	1.000000
max	3.000000

price	0
area	0
bedrooms	0
bathrooms	0
stories	0
mainroad	0
guestroom	0
basement	0
hotwaterheating	0
airconditioning	0
parking	0
prefarea	0

furnishingstatus 0

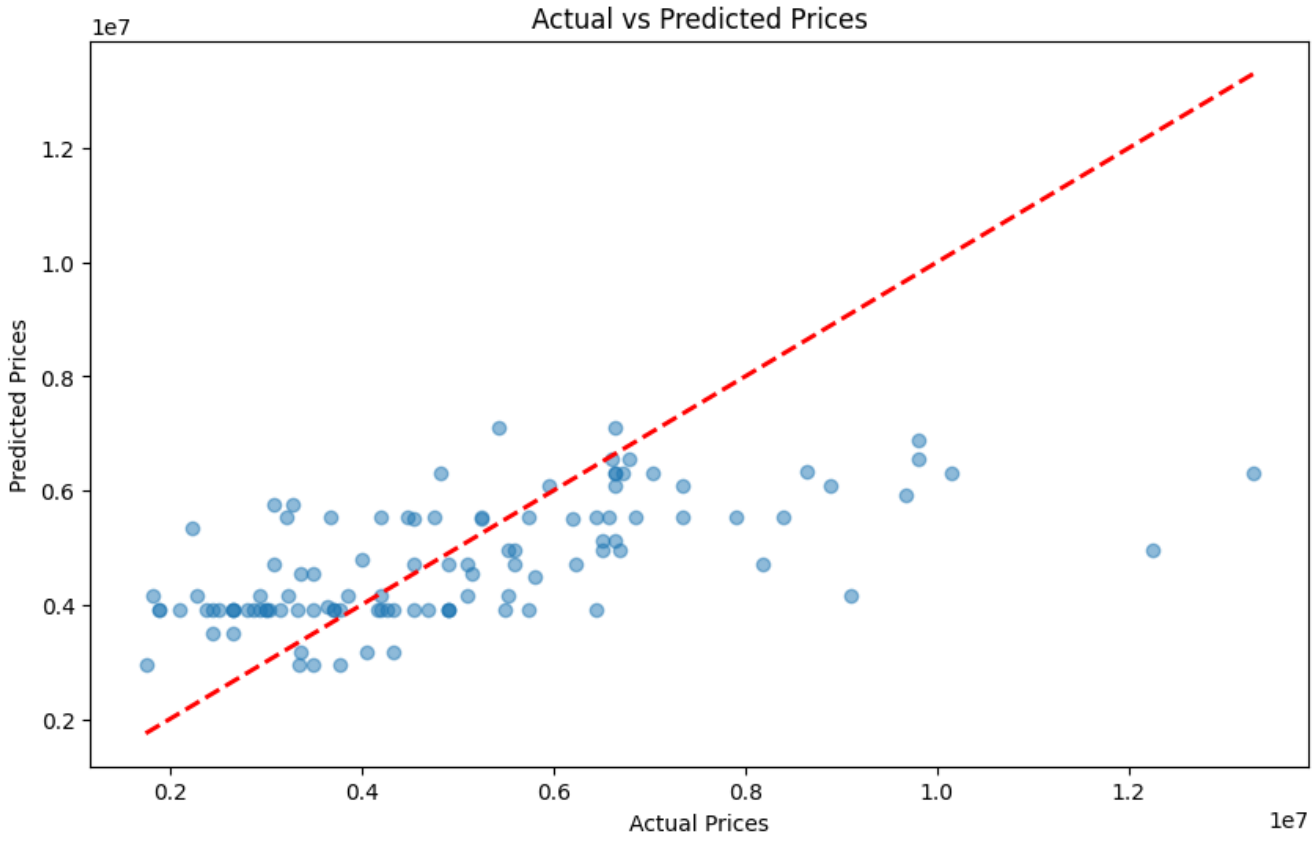
dtype: int64

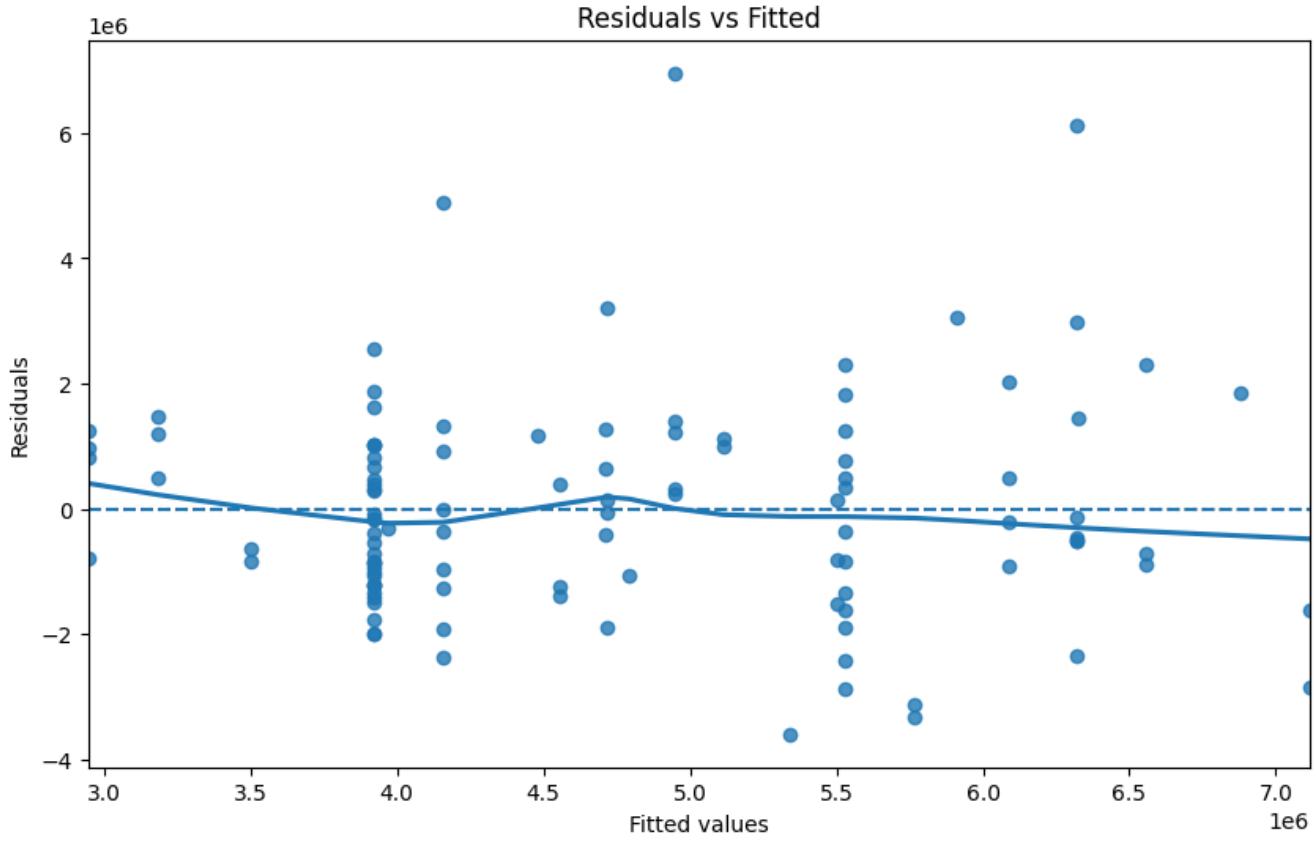




Mean Squared Error: 3254189950926.765

R-squared: 0.35618859916519907


Actual vs Predicted Prices
1e7
Predicted Prices
Actual Prices
1e7
0.2
0.4
0.6
0.8
1.0
1.2


Residuals vs Fitted
1e6
Residuals
Fitted values
1e6
−4
−2
0
2
4
6
3.0
3.5
4.0
4.5
5.0
5.5
6.0
6.5
7.0

Result

The Linear Regression model was successfully implemented to predict house prices based on selected features. The dataset was preprocessed, and the model was trained and tested using a train-test split. The performance of the model was evaluated using Mean Squared Error and R-squared values. Visualization of actual vs predicted values and residual plots indicated how well the model fits the data, showing its effectiveness in predicting house prices.